

Apuntes del Curso: Computación Paralela y Distribuida

Prof. José Fiestas PhD
TA: Marcelo Zuloeta

Abril 2026

Introducción a Message Passing Interface (MPI)

Message Passing es un modelo de comunicación entre **procesos** con memoria local propia, basada en el envío y recepción explícita de mensajes. Este modelo es fundamental para el paralelismo distribuido y posee las siguientes características:

- **Independencia de Topología:** funciona independientemente de la infraestructura de red subyacente (Fat Tree, Torus, Dragonfly, Hypercube, etc.).
- **Espacio de Memoria Particionado:** La memoria se divide entre p nodos.
- **Arquitectura NUMA:** El acceso a la memoria local es significativamente más rápido que el acceso a memoria remota.
- **Versatilidad:** MPI es ejecutable en CPUs individuales, nodos de cómputo o clusters completos, permitiendo paralelismo explícito o híbrido.

De acuerdo con [2] y [3], el modelo de paso de mensajes es el estándar *de facto* para arquitecturas de memoria distribuida. A diferencia del modelo de memoria compartida (OpenMP), aquí cada proceso posee su propio espacio de direccionamiento privado.

Arquitectura y Topología

El estándar MPI es agnóstico a la topología de red subyacente. Los procesos pueden comunicarse a través de infraestructuras complejas como:

- **Fat Tree:** Común en centros de datos para minimizar la latencia.
- **Torus:** Utilizada en supercomputadoras para escalabilidad multidimensional.
- **Hypercube:** Conectividad logarítmica eficiente para algoritmos de reducción.

El estándar MPI (Message Passing Interface)

MPI no es un lenguaje, sino una especificación de biblioteca. Su portabilidad permite que un código escrito en C o C++ se ejecute sin cambios desde una laptop hasta un cluster masivo. La versión más reciente, **MPI-5 (2025)**, optimiza el soporte de arquitecturas híbridas y paralelismo explícito

Evolución y Ecosistema

MPI se estableció como un estándar portable en 1994 (versión 1.0) para permitir que códigos en C, C++ y Fortran funcionen en diversas arquitecturas. Su evolución ha sido constante, alcanzando la versión **MPI-5 en 2025**.

Actualmente, las implementaciones estables más utilizadas son **MPICH** (v4.3.2) y **OpenMPI** (v5.0.9). Además, existen *bindings* para lenguajes de alto nivel como Julia, MATLAB, Python, R y Rust

Estructura y Funciones Básicas

Un programa MPI sigue un flujo de control definido por el ciclo de vida del entorno de ejecución.

Handles y Comunicadores

Los *handles* son referencias a estructuras de datos internas de MPI. El más importante es `MPI_COMM_WORLD`, un objeto de tipo `MPI_Comm` que engloba a todos los procesos activos. Dentro de este comunicador, cada proceso recibe un identificador entero único denominado **rank**, numerado desde 0.

API Fundamental

Las rutinas MPI suelen retornar un entero para la gestión de errores; el valor `MPI_SUCCESS` indica una ejecución correcta.

- `MPI_Init(&argc, &argv)`: Inicializa el entorno. Debe llamarse una sola vez al inicio.
- `MPI_Comm_size(comm, &size)`: Obtiene el número total de procesos.
- `MPI_Comm_rank(comm, &rank)`: Obtiene el ID del proceso actual.
- `MPI_Wtime()`: Retorna el tiempo de reloj en segundos (double).
- `MPI_Finalize()`: Finaliza el entorno y limpia estructuras.

Compilación y Tipos de Datos

La compilación requiere *wrappers* específicos según el lenguaje utilizado.

Lenguaje	Compilador	Comando de Ejemplo
C	mpicc	mpicc -o prog.exe prog.c
C++	mpic++	mpic++ -o prog.exe prog.cpp
Fortran	mpifort	mpifort -o prog.exe prog.f90

Para ejecutar el binario en 4 procesadores, se utiliza: `mpirun -np 4 ./prog.exe`.

Correspondencia de Tipos

MPI define tipos propios para garantizar la comunicación en arquitecturas heterogéneas. Por ejemplo, `MPI_INT` corresponde a `signed int` en C, y `MPI_DOUBLE` a `double`.

El Entorno de Ejecución y Comunicadores

Un concepto central en [4] es el **Comunicador**. Un comunicador define un grupo de procesos que pueden enviarse mensajes entre sí.

MPI_COMM_WORLD

Al iniciar el entorno con `MPI_Init`, se define automáticamente el comunicador global `MPI_COMM_WORLD`.

- **rank:** Es el identificador único del proceso dentro del comunicador
 $rank \in [0, p - 1]$
- **size:** El número total de procesos lanzados por `mpirun` (`size=p`)

`MPI_COMM_WORLD` contiene todos los procesos lanzados por MPI.

Si tenemos $p = 4$, entonces:

$$MPI_COMM_WORLD = \{P_0, P_1, P_2, P_3\}$$

Código y modelo de ejecución MPI

```
1 #include<mpi.h>
2 #include<iostream>
3 using namespace std;
4
5 int main(int argc, char*argv[]){
6 // declaracion de variables
7 int ierr, rank, size, dato=10;
8
9 // esta parte es estandar para los ejercicios de clase
10 ierr=MPI_Init(&argc,&argv);
11 MPI_Comm_rank(MPI_COMM_WORLD,&rank);
12 MPI_Comm_size(MPI_COMM_WORLD,&size);
13
14 // 1. ejecutar la impresion solo si MPI_Init resulto (use MPI_SUCCESS)
15 if(ierr==MPI_SUCCESS)
16 printf("Pekaboo! desde %d de %d con dato: %d\n",rank,size,dato);
17
18 // 2. modificar el valor de datos solo para que el proceso 2 tenga dato
19 // =20
20 if(rank==2) dato=20;
21 // aqui definen las llamadas MPI sobre el secuencial
22 printf("rank %d de %d con dato: %d\n",rank,size,dato);
23 MPI_Finalize();
24 }
```

Al ejecutar

```
1 mpirun -n 4 ./programa
```

MPI crea cuatro procesos:

$$P_0, P_1, P_2, P_3$$

Cada proceso ejecuta el mismo programa, pero puede tomar decisiones distintas usando su identificador o rank.

Si se ejecuta con cuatro procesos se puede obtener:

```

1 Pekaboo! desde 0 de 4 con dato 10
2 Pekaboo! desde 1 de 4 con dato 10
3 Pekaboo! desde 3 de 4 con dato 10
4 Pekaboo! desde 2 de 4 con dato 10

```

El orden de impresión no está determinado debido a que los procesos se ejecutan concurrentemente. El ejemplo Pekaboo (visto en clase) es un SIMD , ya que todos los procesos ejecutan el mismo código, pero trabajan sobre datos diferentes (ranks).

Comunicación Punto a Punto Bloqueante

La comunicación punto a punto es la forma más básica de transferencia de datos. En el modo **bloqueante**, la rutina no regresa hasta que los datos han sido retirados del buffer de la aplicación.

Modos de Transmisión Semántica

Siguiendo la documentación de *MPICH* y las notas de *EPCC*, MPI ofrece cuatro modos para el envío (**Send**):

Modo	Función	Semántica Técnico-Teórica
Standard	<code>MPI_Send</code>	El sistema decide si es sincrónico o buffered según el tamaño del mensaje y el <i>threshold</i> interno
Sincrónico	<code>MPI_Ssend</code>	El envío no finaliza hasta que el receptor ha iniciado la recepción del mensaje. Garantiza sincronía física.
Buffered	<code>MPI_Bsend</code>	Utiliza un buffer de usuario. Requiere <code>MPI_Buffer_attach</code> para asignar memoria explícita.
Ready	<code>MPI_Rsend</code>	Solo se puede llamar si el receptor ya ha posteado su <code>MPI_Recv</code> . De lo contrario, el comportamiento es indefinido (error).

Sintaxis de Comunicación

La forma típica para el envío (`MPI_Send`) y recepción (`MPI_Recv`) requiere los siguientes parámetros:

- **buffer:** Puntero o dirección de la variable de datos.
- **count:** Cantidad de elementos.
- **datatype:** Tipo de dato MPI.
- **dest/source:** Rank del proceso destino o remitente.
- **tag:** Identificador del mensaje para diferenciar múltiples envíos.
- **status:** Objeto que contiene información del remitente y el tag del mensaje recibido.

Gestión del Estado (Status)

A diferencia del emisor, el receptor utiliza un objeto `MPI_Status` para obtener metadatos sobre el mensaje entrante, especialmente útil cuando se usan comodines como `MPI_ANY_SOURCE`.

- `status.MPI_SOURCE`: Indica el rank del remitente.
- `status.MPI_TAG`: Identificador del mensaje.
- `MPI_Get_count`: Función para determinar cuántos elementos del tipo de dato especificado fueron recibidos.

Ejemplo: Intercambio de Datos (1 a 1)

Este código ilustra la lógica de bifurcación por `rank` típica del modelo SPMD (*Single Program Multiple Data*).

```
1 #include <mpi.h>
2 #include <iostream>
3
4 int main(int argc, char* argv[]) {
5     int rank, number;
6     MPI_Init(&argc, &argv);
7     MPI_Comm_rank(MPI_COMM_WORLD, &rank);
8
9     if (rank == 0) {
10         number = -1;
11         // Envio estandar bloqueante
12         MPI_Send(&number, 1, MPI_INT, 1, 0, MPI_COMM_WORLD);
13     } else if (rank == 1) {
14         // Recepcion bloqueante
15         MPI_Recv(&number, 1, MPI_INT, 0, 0, MPI_COMM_WORLD,
16                 MPI_STATUS_IGNORE);
17         std::cout << "Rank_1_recibio:" << number << std::endl;
18     }
19
20     MPI_Finalize();
21     return 0;
22 }
```

Listing 1: Intercambio de un entero entre Ranks 0 y 1

Revise ejemplos de Broadcast y Reduction hechos en clase

1. Bibliografia

Referencias

- [1] <https://www.mpich.org/documentation/guides/>
- [2] William Gropp, Ewing Lusk and Anthony Skjellum: *Using MPI: Portable Parallel Programming With the Message-Passing Interface*. MIT Press, 3rd edition, Nov. 2014 (336 pages, ISBN 9780262527392)

- [3] William Gropp, Torsten Hoefler, Rajeev Thakur and Ewing Lusk: *Using Advanced MPI: Modern Features of the Message-Passing Interface*. MIT Press, Nov. 2014 (392 pages, ISBN 9780262527637)
- [4] Michael J. Quinn. *Parallel Programming in C with MPI and OpenMP*. 1st. McGraw-Hill Education Group, 2003. isbn: 0071232656. Ch 4
- [5] Peter S. Pacheco. *An Introduction to Parallel Programming*. 1st. Morgan Kaufmann, 2011. isbn: 978-0-12-374260- 5. Ch 3